

"END to END ECOMMERCE WEBSITE ANALYSIS"

"Objective:The objective of this analysis is to understand e-commerce sales across different segments and parameters. By examining these dimensions, we gain a clear picture of current performance and identify the actions needed to drive stronger growth.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 20)
```

```
In [2]: ## Reading the CSV files:
```

```
In [3]: df = pd.read_excel("E-Commerce Orders.csv")
df
```

Out[3]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentM
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Deb
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Cred
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Deb
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	
...	...	...	...	...	...	...	...	
1195	ORD201195	2024-06-20	C21126	Desk	1	107.04	392 Main St	Cred
1196	ORD201196	2024-03-04	C20095	Monitor	2	662.53	778 Main St	
1197	ORD201197	2023-07-13	C79674	Tablet	2	436.84	275 Main St	
1198	ORD201198	2024-08-22	C64753	Chair	4	262.52	509 Main St	Deb
1199	ORD201199	2023-06-11	C57502	Tablet	4	560.58	201 Main St	Gil

1200 rows × 14 columns



```
In [4]: df.head(10)
```

Out[4]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMeth
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Ca
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Onl
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Ca
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Ca
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Onl
5	ORD200005	2023-10-23	C37249	Phone	2	245.86	934 Main St	Credit Ca
6	ORD200006	2025-06-17	C83492	Laptop	1	664.42	986 Main St	Gift Ca
7	ORD200007	2023-05-12	C41460	Monitor	5	149.55	706 Main St	Ca
8	ORD200008	2025-04-02	C26817	Phone	2	134.28	904 Main St	Gift Ca
9	ORD200009	2023-11-21	C31946	Desk	4	509.38	102 Main St	Credit Ca



```
In [5]: df.isnull().sum()
```

Out[5]:

OrderID	0
Date	0
CustomerID	0
Product	0
Quantity	0
UnitPrice	0
ShippingAddress	0
PaymentMethod	0
OrderStatus	0
TrackingNumber	0
ItemsInCart	0
CouponCode	309
ReferralSource	0
TotalPrice	0

dtype: int64

```
In [6]: ## Here we need to understand that the coupon code is really needed or not  
df.corr()
```

Out[6]:

	Quantity	UnitPrice	ItemsInCart	TotalPrice
Quantity	1.000000	0.014553	0.650061	0.615251
UnitPrice	0.014553	1.000000	0.000602	0.717081
ItemsInCart	0.650061	0.000602	1.000000	0.392540
TotalPrice	0.615251	0.717081	0.392540	1.000000

```
In [6]: df.groupby("PaymentMethod").count()
```

Out[6]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Or
PaymentMethod								
Cash	246	246	246	246	246	246		246
Credit Card	234	234	234	234	234	234		234
Debit Card	232	232	232	232	232	232		232
Gift Card	230	230	230	230	230	230		230
Online	258	258	258	258	258	258		258

```
In [7]: df1 = df.drop(['TrackingNumber', 'CouponCode'],axis=1,errors = 'ignore')
df1
```

Out[7]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentM
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Deb
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Cred
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Deb
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	
...	...	...	...	...	...	...	...	
1195	ORD201195	2024-06-20	C21126	Desk	1	107.04	392 Main St	Cred
1196	ORD201196	2024-03-04	C20095	Monitor	2	662.53	778 Main St	
1197	ORD201197	2023-07-13	C79674	Tablet	2	436.84	275 Main St	
1198	ORD201198	2024-08-22	C64753	Chair	4	262.52	509 Main St	Deb
1199	ORD201199	2023-06-11	C57502	Tablet	4	560.58	201 Main St	Gil

1200 rows × 12 columns

statstical summary

```
In [8]: df1.describe().T.style.background_gradient(cmap='hot').format('{:,.2f}')
```

```
Out[8]:
```

	count	mean	std	min	25%	50%	75%	max
Quantity	1,200.00	2.95	1.41	1.00	2.00	3.00	4.00	5.00
UnitPrice	1,200.00	356.41	197.18	11.39	186.06	364.21	521.57	699.93
ItemsInCart	1,200.00	5.49	2.28	1.00	4.00	5.00	7.00	10.00
TotalPrice	1,200.00	1,053.97	819.86	11.39	410.52	823.62	1,578.47	3,456.40

When describing the statistical summary, we observe that the standard deviation values are reasonably close to the mean. This suggests the data distribution is normal and that no significant outliers are present.

Unique Values.

```
In [9]: for col in df1.columns:
        print(col,df1[col].is_unique)
```

```
OrderID True
Date False
CustomerID False
Product False
Quantity False
UnitPrice False
ShippingAddress False
PaymentMethod False
OrderStatus False
ItemsInCart False
ReferralSource False
TotalPrice False
```

EDA

How many customers are using each payment method (maximum)?

```
In [10]: df.groupby('PaymentMethod')['CustomerID'].nunique().idxmax()
```

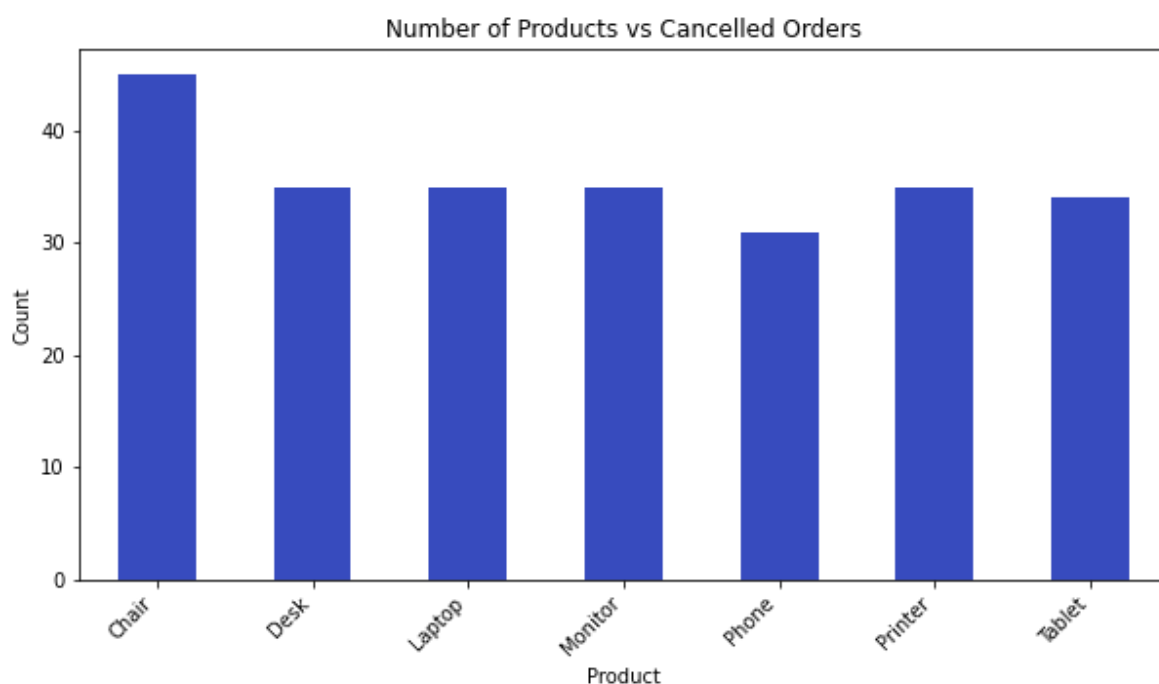
```
Out[10]: 'Online'
```

Maximum Product which get cancelled

```
In [11]: ## The cancelled orders according to the products:
cancelled_df = df1[df1["OrderStatus"]=="Cancelled"]
cancelled_df
cancelled_counts = cancelled_df.groupby("Product")["OrderID"].count()
cancelled_counts
```

```
Out[11]: Product
Chair      45
Desk       35
Laptop     35
Monitor    35
Phone      31
Printer    35
Tablet     34
Name: OrderID, dtype: int64
```

```
In [12]: cancelled_counts.plot(kind="bar",
    colormap="coolwarm",                                     # cycle through colors
    figsize=(10,5)
)
plt.xlabel("Product")
plt.ylabel("Count")
plt.title("Number of Products vs Cancelled Orders")
plt.xticks(rotation=45,ha="right")
plt.show()
```

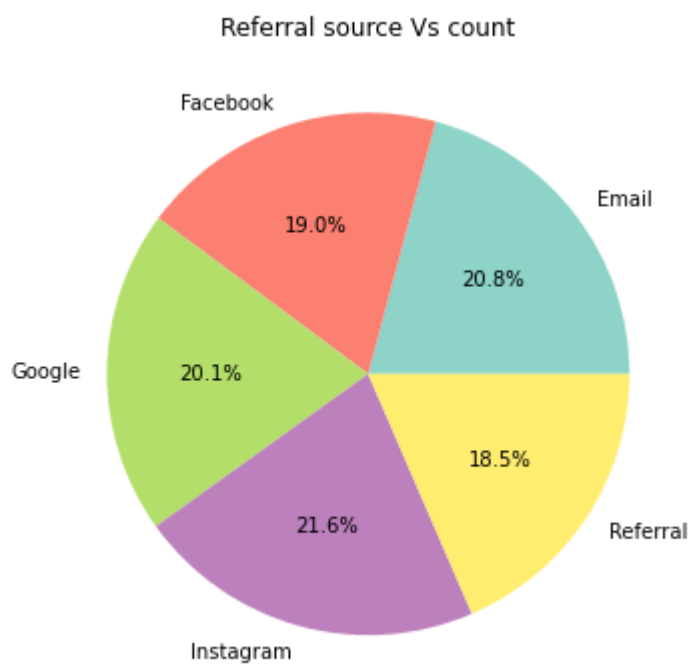


story telling:"The requirement is to identify which product has the highest number of cancellations. From the analysis, the product with the maximum cancellations is Chair. This finding highlights the product most affected by cancellations and helps us understand where issues are concentrated."

Which customers used maximum referral source to buy the product

```
In [13]: ref_counts = df1.groupby("ReferralSource")["OrderID"].count()
ref_percent=(ref_counts/ref_counts.sum())*100
ref_percent.round(2)

## Plotting a pie chart
ref_percent.plot(
    kind = "pie",
    autopct = "%1.1f%%",
    figsize =(6,6),
    colormap= "Set3",
    title = "Referral source Vs count"
)
plt.ylabel(" ")
plt.show()
```



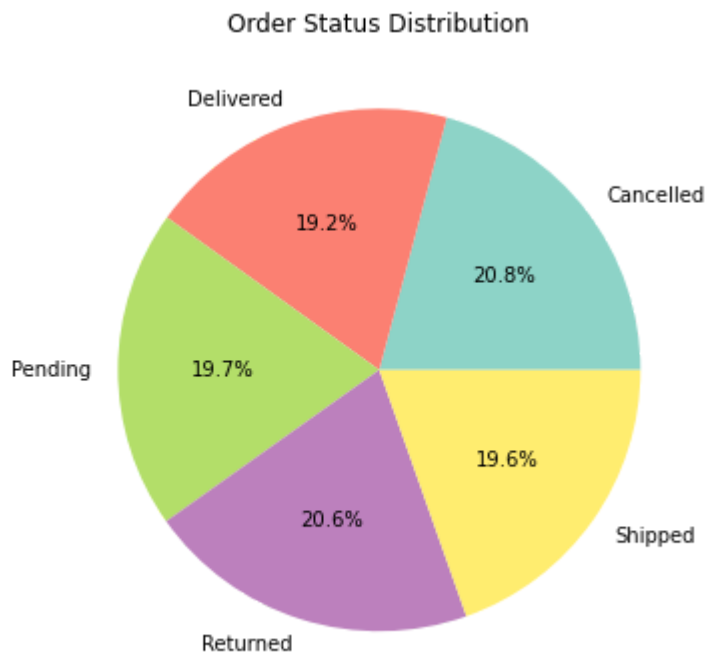
"Storytelling": The objective is to identify where most customers are coming from and which sources generate the highest number of orders. A pie chart was used to visualize this distribution, making it easier to see which channels are strong and which are weak. The analysis shows that referral and Facebook contribute fewer orders, indicating the need to strengthen our Facebook presence and invest in branding and marketing strategies to boost engagement from these channels"

In [14]:

```
ref_counts_order_status = df1.groupby("OrderStatus")["OrderID"].count()
ref_percent_order_status = (ref_counts_order_status / ref_counts_order_status.sum())
print(ref_percent_order_status.round(2))

# Step 4: Plot pie chart
ref_percent_order_status.plot(
    kind="pie",
    autopct="%1.1f%%", # show percentages with one decimal place
    figsize=(6,6),
    colormap="Set3", # adds multiple colors automatically
    title="Order Status Distribution"
)
plt.ylabel("") # cleaner look
plt.show()
```

```
OrderStatus
Cancelled    20.83
Delivered    19.25
Pending      19.75
Returned     20.58
Shipped      19.58
Name: OrderID, dtype: float64
```



"Sorry telling :The pie chart clearly shows that a significant portion of orders were either cancelled or returned. This directly impacts overall sales performance. To address this issue, it is important to analyze the reasons behind cancellations and returns, and to actively track customer feedback. Understanding these patterns will help us refine our sales strategies and reduce the percentage of cancelled and returned orders."

YOY Growth:

```
In [34]: ##Step 1: Make sure Date is datetime
df1["Date"] = pd.to_datetime(df1["Date"])

# Step 2: Extract Year
df1["Year"] = df1["Date"].dt.year

# Step 3: Group by Product and Year
sales_by_product_year = df1.groupby(["Product", "Year"])["TotalPrice"].sum().re
```

```
In [35]: sales_by_product_year
```

```
Out[35]:
```

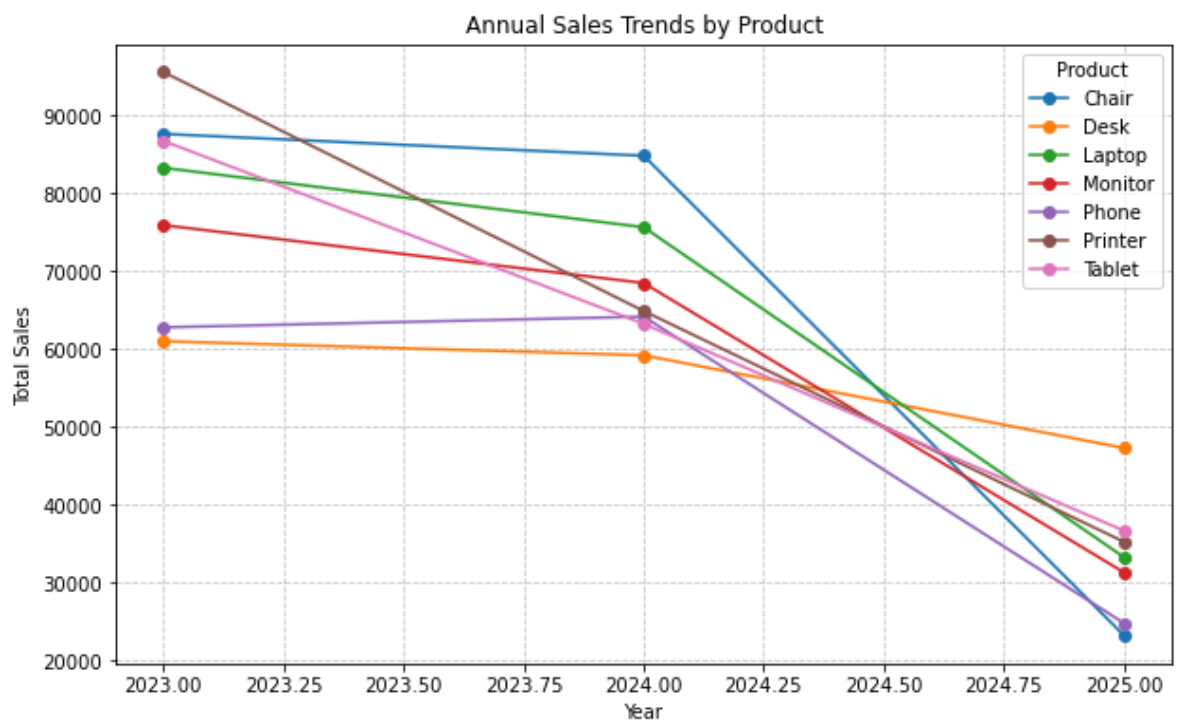
	Product	Year	TotalPrice
0	Chair	2023	87585.58
1	Chair	2024	84785.75
2	Chair	2025	23248.78
3	Desk	2023	60990.53
4	Desk	2024	59177.82
5	Desk	2025	47291.58
6	Laptop	2023	83231.66
7	Laptop	2024	75611.41
8	Laptop	2025	33283.49
9	Monitor	2023	75876.42
10	Monitor	2024	68450.23
11	Monitor	2025	31324.76
12	Phone	2023	62766.40
13	Phone	2024	64143.70
14	Phone	2025	24812.29
15	Printer	2023	95513.73
16	Printer	2024	64853.85
17	Printer	2025	35245.03
18	Tablet	2023	86678.92
19	Tablet	2024	63213.11
20	Tablet	2025	36676.92


```
In [37]: pivot_sales_by_year = df1.pivot_table(
    index="Year",          # rows = years
    columns="Product",     # columns = products
    values="TotalPrice",   # values = sales
    aggfunc="sum"          # sum sales per product per year
)
pivot_sales_by_year
```

```
Out[37]:
```

	Product	Chair	Desk	Laptop	Monitor	Phone	Printer	Tablet
Year								
2023		87585.58	60990.53	83231.66	75876.42	62766.40	95513.73	86678.92
2024		84785.75	59177.82	75611.41	68450.23	64143.70	64853.85	63213.11
2025		23248.78	47291.58	33283.49	31324.76	24812.29	35245.03	36676.92

```
In [43]: pivot_sales_by_year.plot(marker="o", figsize=(10,6))
plt.title("Annual Sales Trends by Product")
plt.xlabel("Year")
plt.ylabel("Total Sales")
plt.legend(title="Product")
plt.grid(True, linestyle="--", alpha=0.7)
# Format y-axis to show integers only
plt.show()
```



```
In [ ]:
```